

Questão 06 - Módulo Terraform no padrão interno

Prompt (aplicando C-A-R-E)

[CONTEXT]

A Hill Valley Tech publicou um padrão interno de IaC (definido por Strickland, head de segurança e compliance) que todo módulo Terraform novo precisa seguir:

- Tags obrigatórias em todo recurso: Owner, CostCenter, Environment
- Prefixo "hvt-" nos nomes de recursos
- Todo bucket S3 deve ter: encryption habilitada (SSE-S3 no mínimo), versioning ativo, block public access total, logging configurado
- Variáveis de entrada em variables.tf com description e type obrigatórios

Doc Brown (CTO) pediu um módulo Terraform reutilizável para criação de buckets S3 aderentes a esse padrão, a ser consumido por todos os times da empresa.

[ACTION]

Crie um módulo Terraform completo para provisionamento de bucket S3, contendo:

1. variables.tf — com todas as variáveis de entrada necessárias (nome do bucket, environment, owner, cost_center, e demais parâmetros configuráveis como habilitar/desabilitar versioning se fizer sentido), cada uma com description e type
2. main.tf — com o recurso aws_s3_bucket e os recursos associados para atender encryption, versioning, block public access e logging (incluindo um bucket de destino para os logs, ou parametrizando um bucket de logging já existente)
3. outputs.tf — expondo ao menos o nome (id) e o ARN do bucket criado
4. Um exemplo de uso do módulo (example/main.tf) mostrando como um time consumidor chamaria esse módulo

Aplique tags via local common_tags (Owner, CostCenter, Environment) mesclada em todos os recursos, seguindo o mesmo padrão usado no módulo de VPC de referência.

[RESULT]

O módulo deve ser aprovável por Strickland sem ressalvas de compliance — ou seja, qualquer bucket criado a partir dele já nasce em conformidade total com o padrão de segurança da empresa, sem exigir configuração manual adicional por parte do time que o consome. O módulo deve ser reutilizável (parametrizado), não um bucket fixo e hardcoded.

[EXAMPLE]

Use como referência de estilo o módulo de VPC já existente na empresa, mantendo a mesma convenção de nomenclatura de variáveis, uso de locals para tags e formato do bloco de tags via merge():

```
variable "environment" {  
    description = "Nome do ambiente (dev, staging, production)"  
    type        = string  
}
```

```
locals {  
    common_tags = {  
        Owner       = var.owner  
        CostCenter   = var.cost_center  
        Environment  = var.environment  
    }  
}
```

```
resource "aws_vpc" "this" {
```

```
cidr_block = var.cidr_block  
tags = merge(local.common_tags, {  
    Name = "hvt-vpc-${var.environment}"  
})  
}
```

[FORMATO DE SAÍDA]

Entregue os 4 arquivos do módulo em blocos de código separados e identificados por nome de arquivo, sem explicações fora dos blocos.

Modelo de IA utilizado

Claude Sonnet 4.5 (Anthropic)

Justificativa do uso

Modelo de referência do curso. Forte aderência a prompts estruturados em múltiplos blocos, raciocínio encadeado consistente e geração de artefatos técnicos (YAML, HCL, bash, SQL) com alto grau de precisão. Escolhido como baseline por permitir avaliação consistente dos frameworks ao longo de todo o desafio.

Output gerado

variables.tf

```
variable "environment" {  
    description = "Nome do ambiente (dev, staging, production)"  
    type       = string  
}  
  
variable "owner" {  
    description = "Time ou pessoa responsável pelo recurso"  
    type       = string  
}  
  
variable "cost_center" {  
    description = "Centro de custo para rateio financeiro do recurso"  
    type       = string  
}  
  
variable "bucket_name" {  
    description = "Nome base do bucket S3 (sem o prefixo hvt-)"  
    type       = string  
}  
  
variable "versioning_enabled" {
```

```
description = "Habilita versionamento no bucket S3"

type      = bool

default   = true

}
```

```
variable "logging_target_bucket" {

  description = "Nome do bucket de destino para armazenar os logs de acesso deste bucket"

  type      = string

}
```

```
variable "logging_target_prefix" {

  description = "Prefixo aplicado aos logs de acesso dentro do bucket de destino"

  type      = string

  default   = "s3-access-logs/"

}
```

main.tf

```
locals {

  common_tags = {

    Owner      = var.owner

    CostCenter = var.cost_center

    Environment = var.environment

  }

  bucket_full_name = "hvt-${var.bucket_name}-${var.environment}"

}
```

```
resource "aws_s3_bucket" "this" {

  bucket = local.bucket_full_name

  tags = merge(local.common_tags, {
```

```
    Name = local.bucket_full_name
  })
}
```

```
resource "aws_s3_bucket_versioning" "this" {
  bucket = aws_s3_bucket.this.id

  versioning_configuration {
    status = var.versioning_enabled ? "Enabled" : "Suspended"
  }
}
```

```
resource "aws_s3_bucket_server_side_encryption_configuration" "this" {
  bucket = aws_s3_bucket.this.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

```
resource "aws_s3_bucket_public_access_block" "this" {
  bucket = aws_s3_bucket.this.id

  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```

```
resource "aws_s3_bucket_logging" "this" {  
  bucket = aws_s3_bucket.this.id  
  
  target_bucket = var.logging_target_bucket  
  target_prefix = var.logging_target_prefix  
}
```

outputs.tf

```
output "bucket_id" {  
    description = "Nome (id) do bucket S3 criado"  
    value      = aws_s3_bucket.this.id  
}
```

```
output "bucket_arn" {  
    description = "ARN do bucket S3 criado"  
    value      = aws_s3_bucket.this.arn  
}
```

example/main.tf

```
module "reactor_events_bucket" {  
    source = "../"  
  
    bucket_name      = "reactor-events"  
    environment      = "production"  
    owner            = "team-reactor"  
    cost_center      = "CC-1042"  
    versioning_enabled = true  
    logging_target_bucket = "hvt-central-logs-production"  
    logging_target_prefix = "s3-access-logs/reactor-events/"  
}
```


Justificativa — como C-A-R-E aparece no prompt

Componente	Onde aparece	Por quê
Context	Padrão de compliance do Strickland (tags, prefixo, requisitos de S3, variables.tf) + quem pediu (Doc Brown) e quem vai consumir (todos os times)	Estabelece as regras não-negociáveis que moldam toda decisão técnica — sem isso a IA poderia gerar um bucket S3 "genérico" sem os hardenings de segurança exigidos
Action	Lista dos 4 arquivos a produzir, com o que cada um deve conter	Traduz o Context em entregáveis concretos e verificáveis, deixando claro que é um módulo (reutilizável) e não um recurso solto
Result	"deve ser aprovável por Strickland sem ressalvas... nasce em conformidade total... reutilizável, não hardcoded"	Define o critério de sucesso da entrega — direciona a IA a parametrizar em vez de fixar valores, e a não deixar nenhum requisito de compliance como "opcional"
Example	Trecho literal do módulo de VPC (variable, locals com common_tags, merge() no tags)	É o componente que garante consistência de estilo entre módulos da empresa — a IA replica a mesma convenção de nomenclatura e estrutura, em vez de aplicar seu próprio estilo genérico de Terraform